FarmFresh Dairy Management System

Complete Project Requirements Document

Generated: 12/8/2026, 10:02:18 am

1. Project Overview

FarmFresh is a full-stack Dairy Farm Management System for farmers, milk collection, customers, subscriptions, inventory, sales challans, deliveries, invoices, payments, and reports. It uses JWT role-based access with ADMIN, SALES, FARMER, and DELIVERY roles.

2. Tech Stack

- Frontend: React, Vite, React Router, Axios, CSS
- Backend: Node.js, Express, TypeScript, Prisma, JWT, bcryptjs, Zod, express-validator
- Database: SQLite (local) / PostgreSQL (production)
- Storage: Amazon S3 for product images (multer-s3 + AWS SDK v3)

3. Role Credentials

- ADMIN !’ admin@farmfresh.com / Admin@123
- SALES !’ sales@farmfresh.com / Sales@123
- FARMER !’ farmer@farmfresh.com / Farmer@123
- DELIVERY !’ delivery@farmfresh.com / Delivery@123

These users are created by prisma/seed.ts with bcrypt-hashed passwords.

4. Sales Challan Flow

- Sales/Admin selects customer and one or more products with quantities.
- System auto-generates challan number: CH-YYYYMMDD-####
- Save as DRAFT (no stock change) or CONFIRMED (stock reduced atomically).
- Status values: DRAFT | CONFIRMED | CANCELLED
- Challan stores product snapshots: name, SKU, unit, price, image URL
- Fields: challan number, customer, products, total quantity, status, created by, created date

5. Business Rules

- Only ADMIN and SALES can create or confirm challans (authorizeRoles middleware).
- Confirming a challan reduces stock inside a Prisma transaction.
- Stock must never become negative.
- Insufficient stock returns HTTP 400 with a clear error message.
- Draft challans can later be confirmed or cancelled.

6. API Endpoints (Sales Challans)

- GET /api/challans — list challans
- GET /api/challans/:id — get challan
- POST /api/challans — create (body: customerId, items[{productId,quantity}], status DRAFT|CONFIRMED)
- POST /api/challans/:id/confirm — confirm draft and reduce stock
- POST /api/challans/:id/cancel — cancel draft

Auth: Bearer JWT. Roles: ADMIN, SALES.

7. Validation Rules

- Mobile: digits only, exactly 10
- Name: alphabets and spaces only
- Email: valid email format
- Price: positive number
- Quantity: positive integer
- SKU: letters, numbers, hyphen only
- Password: min 8 chars, uppercase, lowercase, number

Frontend sanitizes invalid characters, shows inline errors, and disables submit when invalid. Backend uses express-validator and returns JSON: { success:false, message, errors:[{field,message}] }.

8. AWS S3 Integration Steps

- Create S3 bucket and IAM user with s3:PutObject/GetObject on products/*
- Set AWS_REGION, AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_S3_BUCKET_NAME in backend/.env
- Upload via product create/update or POST /api/products/:id/upload
- Allowed: jpg, jpeg, png, webp; max 5 MB
- Store public S3 URL in Product.imagePath; frontend shows thumbnails with placeholder fallback

9. Database Models Summary

- User, Farmer, MilkCollection, Customer, Subscription
- Product, StockMovement, Delivery, Invoice, Payment
- SalesChallan, SalesChallanItem (with product snapshot fields)

10. Deployment Steps

- Configure backend/.env (DATABASE_URL, JWT_SECRET, AWS_*)
- cd backend && npm install && npx prisma migrate deploy && npx prisma generate && npm run seed
- npm run build && npm start (or npm run dev)
- cd frontend && npm install && npm run build && serve dist (or npm run dev)
- Ensure frontend proxy/API base URL points to backend

11. Testing Checklist

- Login works for ADMIN, SALES, FARMER, DELIVERY
- Mobile field rejects alphabets (sanitized + inline error)
- Create challan as DRAFT
- Confirm challan reduces stock
- Confirm with quantity > stock returns HTTP 400
- Product image upload to S3 and thumbnail display
- This PDF exists in project root

12. Viva Explanation

The system separates concerns into React UI, Express REST APIs, and Prisma data access. Role middleware protects routes. Sales challans demonstrate transactional inventory control: drafts do not touch stock; confirmation decrements stock only when every line has sufficient quantity, otherwise the whole transaction rolls back. Validation is enforced on both client and server. Product images are stored on S3 for scalable media handling.